pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Secrets, Limits, and a Cost Meter

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: secrets, limits, and a cost meter

Today

1. One secrets method, end to end
2. secret_names and .env.example
3. Where each platform keeps secrets
4. A leaked token
5. Limits, read today, and cost_meter
6. A live desk's cost, and daily_cap
7. Scheduled work for free

On the course's accounts

Vercel

DESK_MODE set to replay

on the desk service: a setting

GitHub

a scheduled workflow

pushed to the demo repo, run once

Shown as masked transcripts. No cell of the notebook reaches a platform.

One secrets method, in five places

Where the code runs	Where the value lives	How the code reads it
Your laptop	my_bondmath\.env, ignored by Git	`get_secret("NAME", ENV_FILE)`
Colab	the Secrets panel	`get_secret, the same`
Vercel	the project's Environment Variables	`os.environ`
A Space	the Space's variables and secrets	`os.environ`
GitHub Actions	the repository's Actions secrets	`${{ secrets.NAME }}`

```
DESK_MODE live, a key set -> mode live, key loaded yes
```

The code holds a name; the platform holds the value. Nothing typed on a command line.

Every name the code reads

```
deploykit.secret_names(folder)
  os.environ[...]  .get(...)  os.getenv(...)  get_secret(...)
deploykit.env_example(names)
  DESK_MODE=   OPENAI_API_KEY=
```

check_deploy_folder

.env.example: no problems

.env beside it: 2 problems

2

names the desk service reads, both in deploykit.py

2

listed for the bond service: an upper bound

1

HF_TOKEN, in deploy_space.py

Reads code, never a value. A name read and not listed fails on the platform, at run time.

Where each platform keeps secrets

Vercel	Config: readable to members	Secret: write-only once saved
Hugging Face Space	variable: public, copied to a duplicate	secret: private, not copied
GitHub Actions	vars.NAME	secrets.NAME, redacted in logs

Vercel: Config values stay readable; Secret values are write-only once saved (vercel.com/docs/environment-variables, read 2026-10-12).

Vercel: a change to a variable applies only to new deployments (vercel.com/docs/environment-variables, read 2026-10-12).

Spaces: variables are public and copied to a duplicate; secrets are not (huggingface.co/docs/hub/spaces-overview, read 2026-10-12).

GitHub Actions: secrets redacted in logs, not guaranteed (docs.github.com/en/actions/concepts/security/secrets, read 2026-10-12).

The course's own setting, on Vercel

What the platform printed, masked

Recorded 2026-10-12 on the course's own account; account names, URLs, and ids masked to `student`.

```
> vercel env add DESK_MODE production --value replay --type config --yes --project pfi-desk-service
? Added           DESK_MODE
  Project         student/pfi-desk-service
  Environments    Production
  Type            Config
```

```
vercel env ls:   DESK_MODE   <encrypted>   Config   Production   17s   ago
```

```
GET /health:   {"status":"ok","version":"1.0","as_of":"2026-10-02","mode":"replay"}
```

A setting may be typed; a key never is. The variable applies only to the next deployment.

A leaked token

1. **Rotate it on the platform**

   Access Tokens, Manage, refresh or delete: the old value stops working

2. **Change one line**

   the new value after HF_TOKEN= in .env, and in every platform copy

3. **Change no code**

   the code reads the name, and the name did not change

```
lines changed: ['HF_TOKEN']; lines unchanged: ['DESK_MODE']
```

Hugging Face: a leaked token is deleted or refreshed in Access Tokens (huggingface.co/docs/hub/security-tokens, read 2026-10-12).

Limits, read today

platform	what	amount	unit
Vercel Hobby	function invocations a month	1,000,000	invocations
Vercel Hobby	Active CPU a month	4	CPU hours
Vercel Hobby	fast data transfer a month	100	GB
Vercel Hobby	deployments a day	100	deployments
GitHub Actions	minutes, public repository, standard runner	free	minutes
GitHub Actions	minutes a month, private repository, Free plan	2,000	minutes
Hugging Face	ZeroGPU Spaces, free account in good standing	2	Spaces
Hugging Face	ZeroGPU quota a day, free account	5	GPU minutes
Hugging Face	PRO plan, a month	9	US$
Hugging Face Jobs	cpu-basic, an hour (credit balance needed)	0.01	US$

Vercel Hobby: 1,000,000 invocations, 4 CPU hours, 100 GB fast data transfer a month (vercel.com/docs/plans/hobby, read 2026-10-12).

Vercel Hobby: past a usage limit, wait 30 days; non-commercial, personal use only (vercel.com/docs/plans/hobby, read 2026-10-12).

GitHub Actions: free on public repositories (docs.github.com, GitHub Actions billing, read 2026-10-12).

Free means something different on each line. Past a Hobby limit: wait 30 days, no bill, no service.

What a live desk would cost

	exchanges	prompt_tokens	completion_tokens
brief	21	39691	6154
covenant table	6	40810	4216
eval question	32	30406	1463

2026-09	requests	18384.000000	1000000.0	within
2026-09	transfer_gb	0.013379	100.0	within
2026-09	duration_hours	0.074961	4.0	within
2026-09	model_usd	0.000000	NaN	no free tier

US$0.0257

a live /brief: 39,691.0 prompt and 6,154.0 completion tokens

US$0.0015

a live /ask: the eval's average question

US$47.91

September's model cost, live; every platform figure within

The two days of October: US$3.33. In replay, as deployed: US$0 in model requests.

At Course 8's gpt-4.1-mini prices, read 2026-10-08. The platforms would cost nothing; the model, all of it.

A daily cap, and why it is a floor

One process, a cap of 3

`[200, 200, 200, 429, 429, 200]`

The 6th request falls on the next UTC day.

Fake clock: 23:59:58.500, 300 ms a request.

Two copies, each a cap of 3

`[200, 200, 200, 200, 200, 200, 429, 429]`

6 answered against a cap of 3.

Each copy counts its own requests.

```
429  daily cap of 3 requests reached for 2026-10-02 UTC; it resets at 00:00 UTC
```

On a serverless platform the cap is a floor per process, not a guarantee.

Scheduled work for free

```
on:
  schedule:
    - cron: "0 6 * * *"     # 06:00 UTC daily
  workflow_dispatch:          # Run workflow button
jobs: health: runs-on: ubuntu-latest
  - run: curl -sS --fail -o /dev/null -w ... /health
```

The course's run, once

workflow_dispatch on main

log lines: 37; steps: 3

GET /health HTTP status 200

the push: [skip ci], no tests run

```
check_workflow: no pull_request trigger: a pull request is merged without its tests running
check_workflow: no step runs pytest
```

GitHub schedule: UTC, at most every 5 minutes (docs.github.com, Events that trigger workflows, read 2026-10-12).

GitHub schedule: off after 60 days idle in public repos (docs.github.com, Events that trigger workflows, read 2026-10-12).

Excel and Git

Excel side by side (tested in Excel)

`=SUMPRODUCT(model_cost!C2:C5,model_cost!D2:D5)`	47.914342
`=IF(free_tier!B2>free_tier!C2,"over","within")` rows 2 to 4	within, within, within
`=IF(free_tier!B2*10>free_tier!C2,"over","within")` rows 2 to 4	within, within, within

```
.gitignore:3:.env   .env
committed: .env.example (names only), never .env; tag week3
```

Type prices as numbers: a price in a text cell counts as 0 in SUMPRODUCT (day 8).

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the finder knows os.environ[...] and .get(...) but not os.getenv: DESK_MODE is never listed.

Code that runs is not the same as code that is right.

What you can do now

- Keep the value on the platform and the name in the code
- List every name with secret_names; write .env.example
- Rotate a leaked token by changing one line
- Put a month of requests beside the free tiers read today
- Cap a public service's day, and run a check on a schedule

Week 3, Day 5 Capstone: the desk brief as a public app